Traspaso del prototipo CWPP · integrantes 3 y 4

Documento de trabajo complementario al avance del integrante 2. El informe presentable está en `docs/entregables/informe-integrante-2.docx`. Este traspaso reúne decisiones de integración, fuentes y límites de las pruebas.

1. Estado entregado

La VM Ubuntu integra portal, PostgreSQL, Falco y Wazuh manager. Existen alertas reales 100200 (shell detectada por Falco), 100100 (`login_failed`) y 100101 (`access_denied`). Portal y base respondieron saludablemente después de las pruebas. El manager Ubuntu no tiene indexer ni dashboard. El portal de Ubuntu conserva el front anterior; el rediseño se probó por separado en Windows y su fuente está versionada.

Los diagramas están en `docs/evidencias/arquitectura-general.png`, `arquitectura-ubuntu.png` y `arquitectura-windows.png`, con fuentes SVG editables en `docs/`. El avance principal usa el general y el de Ubuntu. El de Windows sirve únicamente para explicar las pruebas históricas en otro entorno.

2. Para el integrante 3: análisis y evaluación

- Relaciona cada afirmación de seguridad con una prueba concreta: captura 27 para Falco → Wazuh real; 28 para auditoría del portal → Wazuh real; 29 para estado y recursos; 26 únicamente para `wazuh-logtest`.
- Usa `docs/guia-analisis-evidencias.md` antes de incorporar otras capturas. Los originales 01–29 están en `docs/evidencias/capturas/` de la PC del proyecto y no se suben a Git. El DOCX de avance incorpora 27–29 como anexo.
- Contrasta las capturas con `docs/evidencias/wazuh-falco-ubuntu-verificacion.json`, `wazuh-portal-ubuntu-verificacion.json`, `metricas-base.json` y `disponibilidad-postgresql.json`. Los JSON están depurados y versionados.
- Trata `docker stats`, `free -h` y `df -h` como muestras puntuales. La última muestra informó 404,8 MiB de Wazuh manager, 2,4 GiB disponibles y 9,0 GB libres; no demuestra rendimiento sostenido.
- Presenta `data_access` como auditoría de consultas emitida por el portal. No hay auditoría de sentencias internas de PostgreSQL. La caída de PostgreSQL y HTTP 503/200 ocurrieron en Windows; 100102/100103 fueron pruebas sintéticas.
- Docker Scout informó un hallazgo alto de zlib en la imagen analizada. El hallazgo no demuestra explotación ni que la imagen reconstruida esté corregida.

3. Para el integrante 4: ensamblaje del informe grupal

- Inserta el avance después del caso, marco teórico, amenazas y requisitos de `Examen Parcial.pdf`. Unifica portada, numeración, citas y referencias con el resto del grupo.
- Conserva las secciones del avance: alcance, solución al problema, arquitectura general y Ubuntu, elementos, relación RF/RNF, resultados y anexos.
- Relaciona RF-01 a RF-06 y RNF-01 a RNF-06 con la matriz de riesgos R01–R06 del examen. Mantén exactamente los estados `Cumplido`, `Parcialmente cumplido` y `No cumplido` según el alcance descrito; no conviertas evidencia parcial en cobertura completa.
- Identifica cada figura por entorno y tipo de prueba. La figura Ubuntu representa cuatro servicios simultáneos. La figura Windows representa evolución histórica y no debe agregarse como componente conectado a la VM.
- Indica que Ubuntu usa Wazuh manager y `alerts.json`; no atribuyas a esa VM un dashboard. La captura 15 de Windows muestra API del manager desconectada.
- El front renovado de Windows es una previsualización de UI. Las capturas de Ubuntu siguen mostrando la interfaz anterior y son válidas para las pruebas de seguridad realizadas allí.

4. Reproducción por otro integrante

Usa un clon del repositorio y sigue `docs/guia-reproduccion.md` y `deploy/ubuntu/README.md`. En el clon Git, la base de Ubuntu es `ubuntu.compose.yaml`; los complementos son `deploy/ubuntu/falco.compose.yaml` y

`deploy/ubuntu/wazuh-manager.compose.yaml`. El ZIP generado por `scripts/paquetizar_ubuntu.py` nombra la base `compose.yaml`. No dejes a Compose elegir automáticamente entre `compose.yml` y `compose.yaml`.

Prepara credenciales locales con `python3 scripts/preparar_ubuntu.py`. Ejecuta `scripts/smoke_portal.py`, `scripts/verificar_falco_json.py`, `scripts/verificar_wazuh_falco.py` y `scripts/verificar_wazuh_portal_ubuntu.py` según la guía. Registra `git log -1 --oneline`, hora UTC y entorno en cada nueva captura. Los scripts de verificación emiten resúmenes breves; evita publicar el contenido completo de los logs.

5. Custodia de evidencia y límites

No publiques `.env`, `credenciales_portal.txt`, certificados, hashes locales, logs completos ni registros personales. Conserva las capturas originales fuera de Git y verifica que no muestren secretos. El JSON de Falco requiere control de tamaño y retención; los permisos actuales no equivalen a un registro inmutable. Detén servicios con Compose sin `down -v` para preservar los volúmenes.

El código, los diagramas y los resultados depurados están versionados. El PDF o DOCX del informe grupal debe reflejar el commit utilizado y mantener separados los hallazgos medidos de las extensiones diseñadas.